PMT1

Schublehre vs. Schublehre

David Koch

david.koch@student.tuwien.ac.at

12503857

26.03.2026

Inhaltsverzeichnis

1 Ansatz **2**
2 Datenbereinigung **2**
2.1 Unbereinigte Standardabweichungswerte 2
2.2 Visualisierung der Ausreißer 3
3 Analyse der bereinigten Messdaten **4**
3.1 Bereinigte Standardabweichungswerte 4
3.2 Histogramm der Messwerte 5
3.3 Statistischer Test 6
4 Fazit **9**

1 Ansatz

Mein erster Gedanke war, die Tabelle an Messdaten mit einem einfachen Python-Skript auszuwerten, um u.a. auf die Standardabweichung der einzelnen Spalten zu kommen.

Noch bevor ich angefangen habe das Skript zu schreiben, fielen mir mit dem bloßen Auge bereits einige ungewöhnliche Datenpunkte auf. Ich habe sie folgend kategorisiert:

1. **Fehlende(r) Messwert(e) (= 0)**

   ```
   20.03.2024 21:02:16 63,549  63,549  0 63,8
   16.03.2025 12:48:33 63  0 0 61,9
   ```

2. **Falsche Einheiten (cm statt mm)**

   ```
   13.03.2025 08:48:57 6,35  6,35  6,39  6,39
   13.03.2025 08:59:27 6,37  6,37  6,4 6,4
   ```

3. **WTF?**

   ```
   27.03.2025 07:11:51 83  45  82  45
   ```

Um zu zeigen, wie sehr solche „Ausreißer“ die statistische Analyse der Messdaten beeinflussen können, habe ich nun eine extra Tabelle erstellt, in welcher diese fünf Datenpunkte nicht enthalten sind. Ich werde diese später mit der Originaltabelle vergleichen.

2 Datenbereinigung

Zur Datenauswertung verwende ich die Python-Bibliothek Pandas. Die erste Iteration des Skripts liest die Daten ein, iteriert durch die Spalten und bildet anhand den Messwerten die Standardabweichung der jeweiligen Spalte:

2.1 Unbereinigte Standardabweichungswerte

```python
import pandas as pd

if __name__ == '__main__':
    df = pd.read_csv('Legosteine_unbereinigt.tsv', sep='\t|;', header=0,
index_col=False, engine="python")
    for colname in df.columns:
        if colname == "Zeitstempel":
            continue
        print(colname)
        df[colname] = df[colname].str.replace(",", ".")
        df[colname] = df[colname].astype(float)
        print(df[colname].std())
```

Quellcode 1: Berechnung der Standardabweichungen mit Pandas (unbereinigt)

Ausgabe:

```
Roter Stein, digitale Schublehre
6.861362504912479
Gelber Stein, digitale Schublehre
8.580384364263951
Roter Stein, analoge Schublehre
10.01214264493199
```

```
Gelber Stein, analoge Schublehre
6.826958813486425
```

Man muss nicht viel über Statistik wissen, um zu merken, dass diese Standardabweichungen für Messwerte, die sich größtenteils im Bereich von 60mm bis 70mm bewegen, etwas schlechtes bedeuten. Besonders der Standardabweichungswert bei der analogen Schublehre für den Roten Stein – welcher bei 10.01214264493199mm liegt – ist sehr groß.

2.2 Visualisierung der Ausreißer

Wenn die Tabellen an Messwerten statt ca. 150 Zeilen um einiges größer wäre, könnte man nicht so einfach per Hand die „Ausreißer“-Messwerte finden und aus dem Datensatz entfernen. Daher ist es hilfreich, einmal alle vorhandenen Daten in einem Diagramm anhand ihrer Häufigkeit zu visualisieren. Daher erstelle ich mit einem Skript nun ein Histogram der Messwerte und markiere mögliche „Ausreißer“ rot:

```python
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

if __name__ == '__main__':
    df = pd.read_csv('Legosteine_unbereinigt.tsv', sep='\t|;', header=0,
index_col=False, engine="python")
    for colname in df.columns:
        if colname == "Zeitstempel":
            continue
        df[colname] = df[colname].str.replace(",", ".")
        df[colname] = df[colname].astype(float)

        data = df[colname].round(1)
        full_range = np.arange(data.min(), data.max() + 0.1, 0.1)

        mean, std = df[colname].mean(), df[colname].std()
        outliers = data[np.abs(data - mean) > std]
        normal = data[np.abs(data - mean) <= std]

        plt.hist(data, bins=full_range)
        plt.yscale("log")
        plt.vlines(outliers, ymin=-1, ymax=1, color='red', linewidth=2)
        plt.title(colname)
        plt.xlabel("Gemessene Länge [mm]")
        plt.ylabel("Anzahl Messungen")
        plt.show()
```

Quellcode 2: Generierung der Ausreißerdiagramme

Ausgabe:

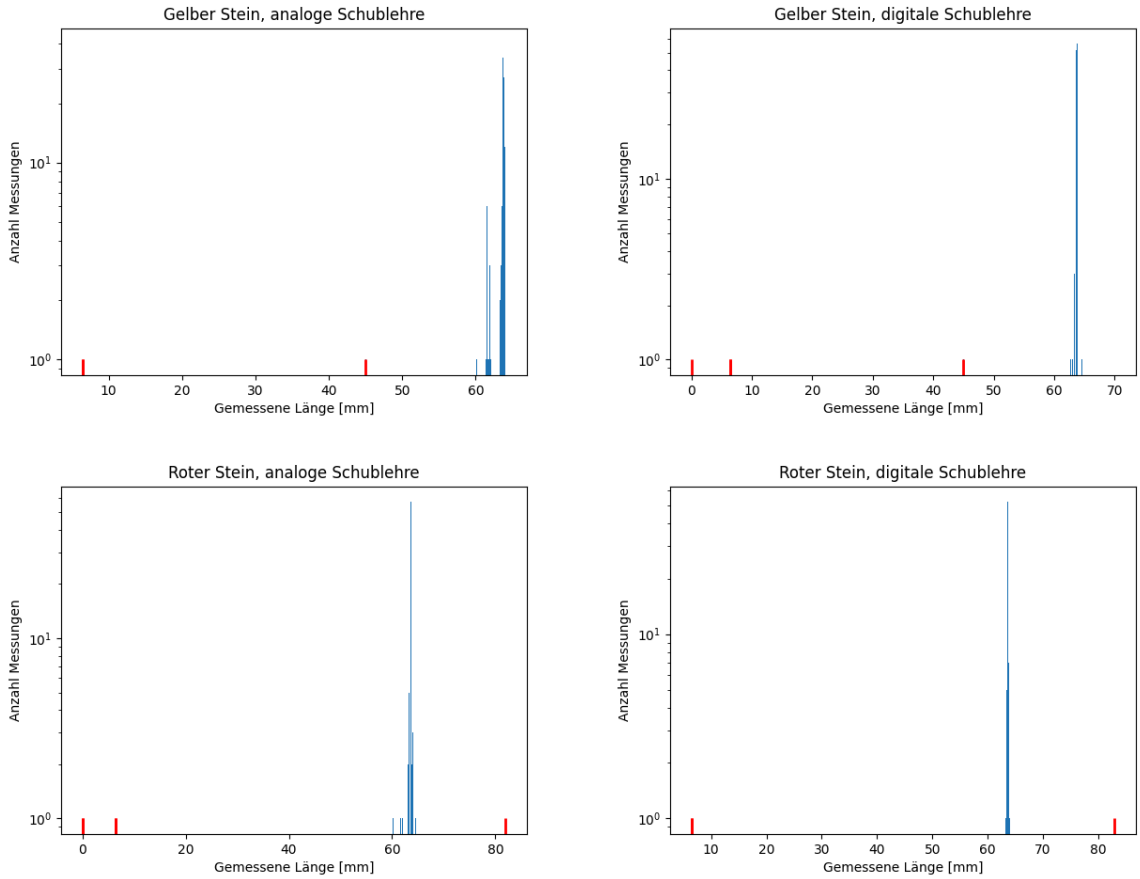


Abbildung 1: Ausreißervisualiserung anhand von Histogrammen (links analog, rechts digital)

Die Erkennung der „Ausreißer“ im obigen Skript ist sehr einfach gehalten, es werden alle Messwerte ausserhalb des Intervalls $[\overline{x} - s; \overline{x} + s]$ als solche gekennzeichnet. Es wurde ebenfalls die y-Achse logarithmisch skaliert, um die Werte mit nur jeweils wenig Messungen besser zu visualieren. Für größere Datenmengen müsste die Datenbereinigung automatisiert ablaufen, da die ca. 150 Zeilen jedoch überschaubar sind, habe ich es mir erlaubt diese manuell zu entfernen. Ob die fehlerhaften Datenpunkte oder die Bereinigung dieser eine Fälschung der Messwerte darstellen lässt sich diskutieren. Ich werde ab diesem Punkt jedoch nur mehr mit den bereinigten Daten arbeiten, da diese dem Schublehrenvergleich besser dienen.

3 Analyse der bereinigten Messdaten

3.1 Bereinigte Standardabweichungswerte

Wenn man das erste Skript (Quellcode 1) nun auf dem bereinigten Datensatz ausführt, ergeben sich direkt viel realistischere Standardabweichungen:

```
Roter Stein, digitale Schublehre
0.24450917526655666
Gelber Stein, digitale Schublehre
0.5866822047664915
Roter Stein, analoge Schublehre
0.6629643145745936
```

```
Gelber Stein, analoge Schublehre
0.6767382602305372
```

Es kann auch schon ein erster ordentlicher Unterschied zwischen den Messungen der digitalen und der analogen Schublehre erkannt werden, denn die Standardabweichungen bei der analogen Schublehre sind größer als die ihrer digitalen Gegenstücke.

Der Grund, wieso der Wert für den gelben Stein mit der digitalen Schublehre um ein Vielfaches größer ist als der für den roten Stein, ist, weil folgender Messwert existiert:
`14.03.2024 15:17:14 63  70  63  61,5`

Ich würde jedoch behaupten, dass es sich hier nur um eine unordentlich durchgeführte Messung handelt und keinen expliziten Datenfehler, wie z.B. verwechselte Einheiten oder Nullwerte (siehe Abschnitt 1). Daher ist es wichtig, diesen Messwert in der statistischen Auswertung zu inkludieren, da er zeigt, dass menschliche Fehler bei der Messung zu Abweichungen führen können, egal ob mit der analogen oder der digitalen Schublehre.

3.2 Histogramm der Messwerte

Wenn man den bereinigten Datensatz wieder per Matplotlib als Histogramm visualisert – diesmal ohne einer besonderen Kennzeichnung der Ausreißer – sehen diese wie folgt aus:

```python
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

if __name__ == '__main__':
    df = pd.read_csv('Legosteine_bereinigt.tsv', sep='\t|;', header=0,
index_col=False, engine="python")
    for colname in df.columns:
        if colname == "Zeitstempel":
            continue
        df[colname] = df[colname].str.replace(",", ".")
        df[colname] = df[colname].astype(float)

        data = df[colname].round(1)
        full_range = np.arange(data.min(), data.max() + 0.1, 0.1)

        plt.hist(data, bins=full_range)
        plt.title(colname)
        plt.xlabel("Gemessene Länge [mm]")
        plt.ylabel("Anzahl Messungen")
        plt.show()
```

Quellcode 3: Generierung der Histogramme

Ausgabe:

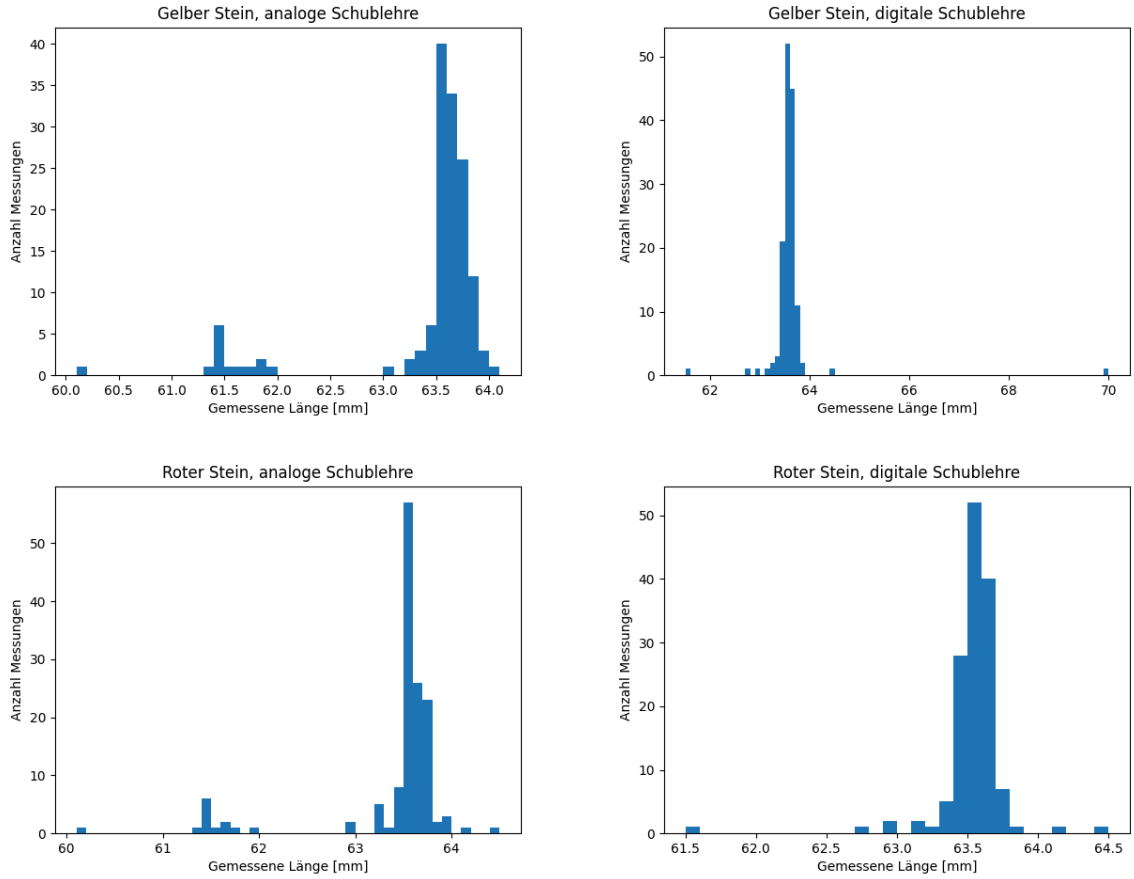


Abbildung 2: Histogramme der Schublehrenmesswerte (links analog, rechts digital)

In den Diagrammen lässt sich jetzt schon eine Art Normalverteilung erkennen. Bei der analogen Schublehre gibt es für den roten als auch den gelben Lego-Stein einige Messwerte, die ein paar Millimeter unter dem Durchschnitt liegen. Meine Vermutung für den Ursprung dieser Messwerte sind menschliche Fehler bei dem Ablesen der Längenmessung auf der analogen Schublehre.

3.3 Statistischer Test

Nun kommt der wirkliche statistische Test der beiden Schublehren:

Null-Hypothese H_0: Die beiden Schublehren messen gleich (Durchschnitt x = Durchschnitt y)

Ich werde diesen Test für beide Steine durchführen, beginnend mit dem roten Stein. Für die Implementierung des Tests in Python habe ich die Bibliothek SciPy verwendet, da diese einfach die Formeln für die T-Statistik und den p-Wert abstrahiert und direkte Ergebnisse liefert.

```python
import pandas as pd
from scipy.stats import ttest_ind

if __name__ == '__main__':
    df = pd.read_csv('Legosteine_bereinigt.tsv', sep='\t|;', header=0,
index_col=False, engine="python")
    columns_for_ttest = []

    for colname in df.columns:
        if "Roter Stein" not in colname:
            continue
        print(colname)
        columns_for_ttest.append(colname)
        df[colname] = df[colname].str.replace(",", ".")
        df[colname] = df[colname].astype(float)

        mean, std = df[colname].mean(), df[colname].std()
        print(f"Mittelwert: {mean}")
        print(f"Standardabweichung: {std}")
        print()

    t_test = ttest_ind(df[columns_for_ttest[0]], df[columns_for_ttest[1]])
    print(f"T-Statistik: {t_test[0]}")
    print(f"p-Wert: {t_test[1]}")
```

Quellcode 4: Berechnung T-Statistik und p-Wert mittels SciPy

Ausgabe (Roter Stein):

```
Roter Stein, digitale Schublehre
Mittelwert: 63.58802816901409
Standardabweichung: 0.24450917526655666

Roter Stein, analoge Schublehre
Mittelwert: 63.45862676056339
Standardabweichung: 0.6629643145745936

T-Statistik: 2.182225371823352
p-Wert: 0.02991809887877914
```

Ausgabe (Gelber Stein):

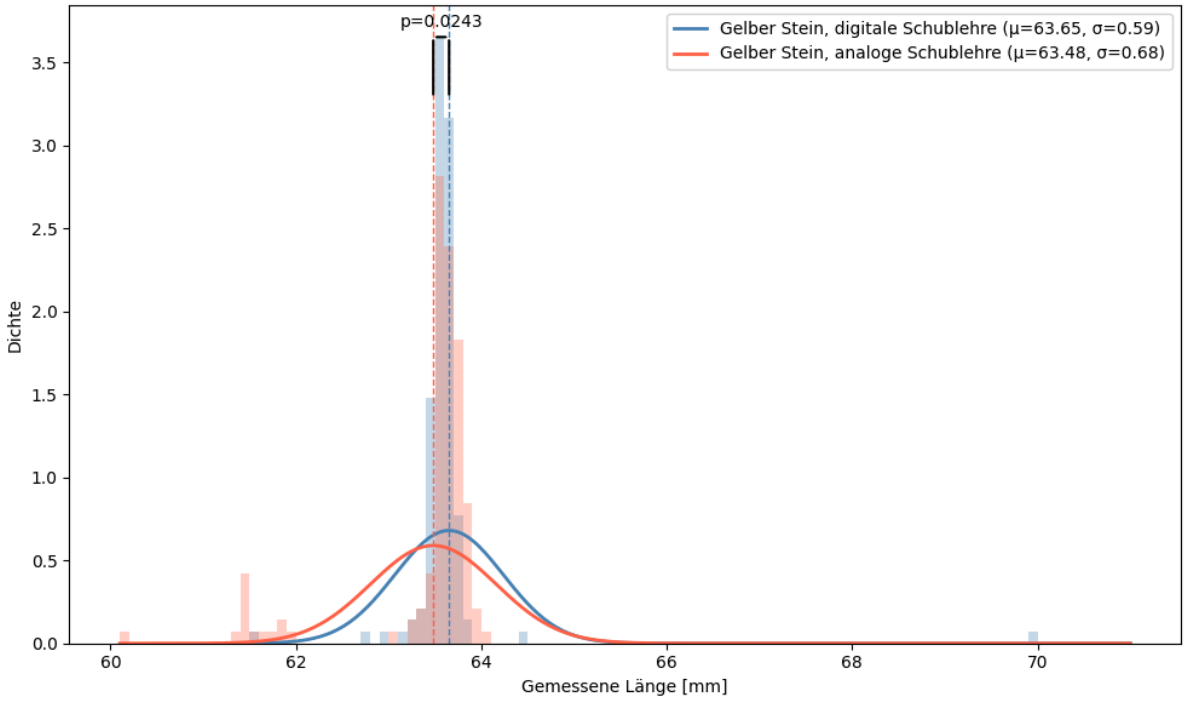
```
Gelber Stein, digitale Schublehre
Mittelwert: 63.6524647887324
Standardabweichung: 0.5866822047664915

Gelber Stein, analoge Schublehre
Mittelwert: 63.48225352112676
Standardabweichung: 0.6767382602305372

T-Statistik: 2.264639539165988
p-Wert: 0.02429394033817604
```

Laut der Vorlesung wird die Nullhypothese, dass beide Schublehren gleich messen, verworfen, wenn $p < 0.05$ ist. Da für beide Steine der p-Wert sogar kleiner als 0.3 ist, wird die Nullhypothese somit verworfen. Beide Schublehren messen nicht gleich, und da die Standardabweichungen bei der digitalen Schublehre niedriger sind als bei ihrem analogen Gegenstück, würde ich behaupten, dass diese genauere Messungen liefert.

Die Ergebnisse des statistischen Tests lassen sich natürlich auch noch visualisieren:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import ttest_ind, norm

if __name__ == '__main__':
    df = pd.read_csv('Legosteine_bereinigt.tsv', sep='\t|;', header=0,
index_col=False, engine="python")
    columns_for_ttest = []

    fig, ax = plt.subplots(figsize=(10, 6))
    colors = ["steelblue", "tomato"]

    for color, colname in zip(colors, [c for c in df.columns if "Roter Stein" in c]):
        columns_for_ttest.append(colname)
        df[colname] = df[colname].str.replace(",", ".").astype(float)

        data = df[colname].round(1)
        mean, std = df[colname].mean(), df[colname].std()
        full_range = np.arange(data.min(), data.max() + 0.1, 0.1)

        ax.hist(data, bins=full_range, density=True, alpha=0.3, color=color)
        x = np.linspace(data.min(), 71, 300)
        ax.plot(x, norm.pdf(x, mean, std), color=color, linewidth=2,
label=f"{colname} (μ={mean:.2f}, σ={std:.2f})")
        ax.axvline(mean, color=color, linestyle="--", linewidth=1)

    t_test = ttest_ind(df[columns_for_ttest[0]], df[columns_for_ttest[1]])
    p = t_test[1]

    y_max = ax.get_ylim()[1]
    x1 = df[columns_for_ttest[0]].mean()
    x2 = df[columns_for_ttest[1]].mean()
    bracket_y = y_max * 0.95

    ax.annotate("", xy=(x2, bracket_y), xytext=(x1, bracket_y),
arrowprops=dict(arrowstyle="-", color="black", lw=1.5))
    ax.annotate("", xy=(x1, bracket_y), xytext=(x1, bracket_y * 0.9),
arrowprops=dict(arrowstyle="-", color="black", lw=1.5))
    ax.annotate("", xy=(x2, bracket_y), xytext=(x2, bracket_y * 0.9),
arrowprops=dict(arrowstyle="-", color="black", lw=1.5))

    ax.text((x1 + x2) / 2, bracket_y * 1.01, f"p={p:.4f}", ha="center", va="bottom",
fontsize=10)

    ax.set_xlabel("Gemessene Länge [mm]")
    ax.set_ylabel("Dichte")
    ax.legend()
    plt.tight_layout()
    plt.show()
```

Quellcode 5: Visualiserung der Gaußverteilungen inkl. p-Werte (für Rot)



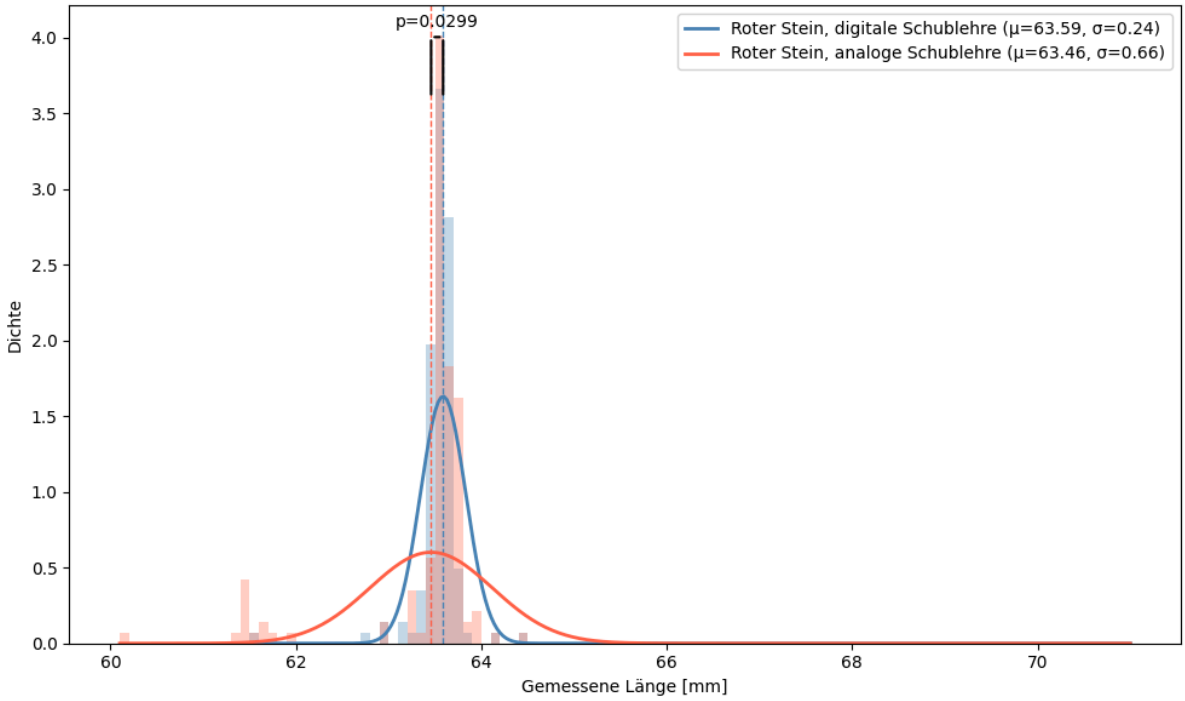


Abbildung 3: Diagramme zum Vergleich der Gaußverteilungen

4 Fazit

Eine wichtige Bemerkung am Ende ist, dass die p-Werte für die unbereinigten Datensätze bei weit über 0.05 liegen, nämlich 0.3224468176848559 für den roten und 0.774912406669791 für den gelben Stein. Das heißt, hätte man nicht die Daten bereinigt, wäre die Nullhypothese fälschlicherweise nicht verworfen worden.

Für den bereinigten Datensatz waren die p-Werte beide kleiner als 0.05, wodurch die Nullhypothese, dass beide Schublehren gleich messen, verworfen. Aufgrund der geringeren Sträuung bei den Messwerten der digitalen Schublehre würde ich behaupten, dass diese genauere Messwerte liefert als ihr analoges Gegenstück. Dies lässt sich wahrscheinlich damit begründen, dass das korrekte Ablesen von der gemessenen Länge auf einer analogen Schublehre für unerfahrene Leute verwirrend sein kann, was zu Ungenauigkeit führt.